

Optimization-Based GNSS Receiver Positioning Using Weighted Least Squares

Project Documentation

Asja Smjecanin
Faculty of Electrical Engineering (ETF), Sarajevo
Index: 20012

September 18, 2026

1 Introduction

The goal of this project is to implement and compare a few numerical optimization methods that are used for solving the classical GNSS (Global Navigation Satellite System) receiver positioning problem. The project problem is: given a set of satellites with known positions and measured pseudoranges (noisy distances) to the receiver, we want to estimate the receiver's unknown position and clock bias.

This is a nonlinear least squares problem, because the distance from receiver to satellite is a nonlinear function of the receiver's coordinates. In this project we implemented three iterative methods for solving it:

- **Gauss-Newton (GN)** - approximates the Hessian using only first-order (Jacobian) information.
- **Gradient Descent (GD)** - uses only the gradient direction, with a simple backtracking line search.
- **Full Newton** - uses the exact Hessian, including second-order (curvature) terms.

The project also uses a weighted least squares formulation, where each satellite's residual is weighted depending on how reliable its measurement is assumed to be. We compare what happens when the assumed weighting strategy matches the real (physical) noise model, and what happens when it does not.

For the linear algebra (solving the normal equations at each iteration) we used the sparse solver from the `natID` framework given by the professor, and for the whole GUI/visualization part we also used `natID`'s GUI module (`gui:::`).

The application shows, in real time, a sky-plot of the satellites, a convergence plot of how the estimated position moves towards the true position, and a cost-vs-iteration chart, for all three algorithms.

2 Problem Formulation

2.1 State vector

The receiver state is a 4-dimensional vector: 3D position plus clock bias b (in meters, since clock bias is converted to a distance by multiplying with the speed of light):

$$\mathbf{x} = [X \ Y \ Z \ b]^T$$

2.2 Measurement model

For satellite i with known position (X_i, Y_i, Z_i) , the predicted pseudorange is:

$$\hat{\rho}_i(\mathbf{x}) = \sqrt{(X - X_i)^2 + (Y - Y_i)^2 + (Z - Z_i)^2} + b$$

The residual for satellite i is the difference between the measured pseudorange ρ_i and the predicted one:

$$r_i(\mathbf{x}) = \rho_i - \hat{\rho}_i(\mathbf{x})$$

2.3 Weighted cost function

We minimize a weighted sum of squared residuals:

$$F(\mathbf{x}) = \frac{1}{2} \sum_{i=1}^n w_i r_i(\mathbf{x})^2$$

where w_i is the weight assigned to satellite i . This is exactly what the `costAt()` function computes in code.

2.4 Jacobian

The Jacobian matrix is computed using each satellite as one row. For satellite i (with respect to X, Y, Z, b), the row of the Jacobian is computed as its geometric unit vector, plus 1 for the clock bias term:

$$J_i = \begin{bmatrix} \frac{X - X_i}{d_i} & \frac{Y - Y_i}{d_i} & \frac{Z - Z_i}{d_i} & 1 \end{bmatrix}$$

where d_i is the current predicted geometric distance to satellite i .

2.5 Normal equations (Gauss-Newton)

For Gauss-Newton, we build the weighted normal equations:

$$G = J^T W J, \quad \text{rhs} = J^T W r$$

and solve $G \Delta \mathbf{x} = \text{rhs}$ for the update step $\Delta \mathbf{x}$. This is exactly what `buildNormalEquations()` builds, one satellite at a time (accumulating into a 4×4 matrix), and it is solved with `natID`'s sparse solver, not with a manually written linear solver.

2.6 Full Newton correction

The full Newton method adds the second-order curvature term to the Gauss-Newton approximation. For each satellite, the extra Hessian contribution is:

$$H_i = -w_i r_i \cdot \frac{I - J_{1:3,i} J_{1:3,i}^T}{d_i}$$

which is added on top of G . This term becomes negligible when the residuals r_i are small compared to the distance d_i - and since GPS slant ranges are around 2×10^7 m, this term almost never matters in practice. That is why, in our tests, full Newton behaved really similarly to Gauss-Newton, but we still wanted to show it.

2.7 Gradient descent with backtracking

Gradient descent takes a step in the direction of $\text{rhs} = J^T W r$ (which is the negative gradient direction of F), with step size α chosen by simple backtracking: start from some α , and halve it until the cost actually decreases (using the bisection method from the course materials).

2.8 Weighting strategies

Three weighting strategies were implemented:

- **Uniform:** $w_i = 1$ for all satellites.
- **TrueElevation:** $w_i = 1/\sigma_{\text{true},i}^2$, which matches the physically simulated noise model exactly.
- **ElevationSquared:** assumes noise grows like $\sigma_0/\sin^2(\text{elevation})$ instead of the true $\sigma_0/\sin(\text{elevation})$ - this is deliberately a *wrong* model, used to show what happens when the assumed weighting does not match reality.

2.9 Dilution of Precision (DOP)

To quantify how good the satellite geometry is (independent of the noise itself), we compute PDOP (position) and GDOP (geometry, including clock bias) from the diagonal of the inverse of G :

$$\text{PDOP} = \sqrt{Q_{XX} + Q_{YY} + Q_{ZZ}}, \quad \text{GDOP} = \sqrt{Q_{XX} + Q_{YY} + Q_{ZZ} + Q_{bb}}$$

where $Q = G^{-1}$. In code, this inverse is computed by hand with Gauss-Jordan elimination on an augmented 4×8 matrix (function `computeDOP()`), because we only need the diagonal of a small 4×4 inverse, so calling the full sparse solver again for this one thing was not necessary.

3 Project Architecture

The project is a small desktop GUI application, built on top of the professor's `natID` framework (both the `sparse::` module for linear algebra and the `gui::` module for the interface). The source is split as follows:

- `GNSSModel.h` - the actual math: satellite simulation, cost function, the three solvers, DOP.
- `ViewGNSS.h` - a custom `gui::Canvas` that draws the sky-plot, convergence plot, and cost chart, and also drives the step-by-step animation.

- `MainView.h` - the control panel (sliders, combo boxes) that lets the user change parameters.
- `MainWindow.h`, `MenuBar.h`, `Application.h`, `main.cpp` - where we actually start a `natID` GUI app.

4 GNSSModel.h

In this script is the optimization logic. The class `GNSSModel` keeps a vector of `Satellite` structs (each one has position, true noise sigma, assumed weight, pseudorange, and azimuth/elevation for drawing), plus the true receiver position `_truePos`.

4.1 generateConstellation()

Randomly places `nSat` satellites around the true receiver position, using random azimuth and elevation (elevation is restricted above the elevation mask, by default 15° , because the satellites which are too low on the horizon are normally not used in real GNSS receivers). For each satellite, the "true" noise sigma is computed from elevation ($\sigma_{\text{true}} = \sigma_0 / \sin(\text{el})$ - lower elevation means noisier measurement. That actually matches physical reality since the signal passes through more atmosphere), then a noisy pseudorange is generated using a normal distribution with that sigma. This is the ground truth noise, independent from whatever weighting strategy the solver assumes.

4.2 buildNormalEquations() and gaussNewtonStep()

`buildNormalEquations()` loops over all satellites and accumulates G and rhs as described in Section 2.5. Then `gaussNewtonStep()` takes this 4×4 system and solves it with the `natID` sparse solver:

```

1 sparse::DblSolverReleaser p_solver(
2     sparse::createDblSolver(4, 10, sparse::Symmetry::SymmetricPosDef,
3                             sparse::SolverType::LDLT)
4 );
5 sparse::DblSolver& solver = *(p_solver.ptr());
6 for (int a = 0; a < 4; ++a)
7     for (int b = a; b < 4; ++b)
8         solver.addTriple(a, b, G[a][b]);
9 for (int a = 0; a < 4; ++a)
10     solver.setRHS(a, rhs[a]);
11 if (!solver.factorize()) return GNSSState{};
12 solver.solve();

```

We use LDLT factorization, since G is symmetric positive definite by construction (it's a sum of $J^T W J$ terms). We build G and rhs manually with our own loops instead of using `natID`'s built-in `IMatrix::calcGainAndRHS`, because that call was crashing on our setup - so we only rely on `natID` for the actual factorization/solve step, not for assembling the system.

4.3 gradientDescentStep()

Reuses `buildNormalEquations()` just to get the gradient (rhs), ignores G , then does the backtracking line search described in Section 2.6, halving α up to `maxBacktrack` times until the cost decreases.

4.4 buildNewtonSystem() and newtonStep()

Same idea as Gauss-Newton, but adds the curvature correction from Section 2.5 into the Hessian matrix. Since the resulting matrix is not guaranteed positive-definite anymore (it's only symmetric), the solver is created with `Symmetry::SymmetricIndef` instead of `SymmetricPosDef`. If factorization fails (e.g. indefinite/singular system), it falls back to a plain Gauss-Newton step - this is the `usedFallback` flag.

4.5 runGaussNewton() / runGradientDescent() / runNewton()

These are just the iteration loops: repeat the corresponding step function until the step norm is smaller than a tolerance (or, for GD, until the cost stops improving), and record the full history of states and costs at each iteration. This history is later used both for the animation and for the cost-vs-iteration chart.

4.6 computeDOP()

Builds an augmented 4×8 matrix $[G \mid I]$ and reduces it to $[I \mid G^{-1}]$ using Gauss-Jordan elimination with partial pivoting, then reads PDOP/GDOP off the diagonal as in Section 2.7. Returns `false` if the pivot becomes numerically singular (this happens with too few satellites, e.g. bad geometry with only 4 satellites almost collinear).

5 ViewGNSS.h - Visualization

This class inherits from `gui::Canvas` (part of `natID`'s GUI module) and is responsible for both running the model and drawing everything.

5.1 Layout

The canvas is split into three panels using a small helper struct `PanelBounds`: sky-plot (left), convergence plot (right), and cost chart (bottom strip). The panel boundaries are recomputed every frame from the current canvas size (`skyPlotBounds()`, `convergenceBounds()`, `chartBounds()`), so the layout is fully responsive to window resizing.

5.2 rerun()

Whenever any parameter changes (satellite count, noise level, weighting, target/guess position), `rerun()` is called: it re-runs all three algorithms from scratch (`runGaussNewton`, `runGradientDescent`, `runNewton`), resets the animation to step 0, and recomputes DOP for the final Gauss-Newton state.

5.3 Sky-plot drawing (skyPlotPoint())

Draws each satellite as an orange marker, positioned by azimuth/elevation using a standard polar-to-cartesian sky-plot projection: elevation controls the radius (zenith = center, horizon = edge), azimuth controls the angle:

$$r = \frac{90^\circ - \text{el}}{90^\circ} \cdot r_{\max}, \quad x = x_c + r \sin(\text{az}), \quad y = y_c - r \cos(\text{az})$$

5.4 Convergence plot (`convergencePoint()`)

Shows the true position (teal marker), the initial guess (red marker), and a trail of the estimate as it converges, in the active algorithm's color (teal for GN, orange for GD, green for Newton).

5.5 `drawCostChart()`

Plots $\log_{10}(\text{cost})$ vs. iteration number, for all three algorithms overlaid, with the active one drawn thicker. Log scale is needed because the cost typically drops by several orders of magnitude within a few iterations, so a linear scale would make the later iterations invisible.

5.6 Step animation

`_currentStep` advances automatically over time (paced by `_stepTimer`), through play/pause/restart controls (`play()`, `pause()`, `restartPlayback()`), letting the user watch the estimate move step by step toward the true position rather than just seeing the final result.

5.7 `runBatchExperiment()`

Runs `nTrials` independent noisy realizations of the same scenario (same satellite geometry, new random noise each time), records the final position error of all three algorithms, and reports mean $\pm$ standard deviation for each. This gives a statistical comparison instead of relying on a single lucky/unlucky run.

6 GUI Wiring (MainView, MainWindow, MenuBar, Application)

6.1 MainView.h

Builds the control panel using `natID`'s `gui::GridLayout + gui::GridComposer`: combo boxes for algorithm choice, weighting strategy, and playback; sliders for satellite count, noise level, target position offset, and initial guess offset. Each control has an `onChangedValue/onChangedSelection` callback that forwards the new value into `ViewGNSS`.

6.2 MainWindow.h

Sets up the window, attaches the menu bar and main view, and handles the two menu actions ("Reset" and "Run 50 Trials") by forwarding them to the corresponding `ViewGNSS` method.

6.3 MenuBar.h / Application.h / main.cpp

Standard `natID` application entry point: `Application::createInitialWindow()` creates the `MainWindow`, and `main.cpp` just constructs the `Application` and calls `run()`.

7 Summary of natID Usage

To be clear about where exactly `natID` was required and used:

- **Sparse solver** (`sparse::DblSolver`, LDLT factorization) - used in `gaussNewtonStep()` and `newtonStep()` to solve the 4×4 normal equations at every iteration.

- **GUI framework** (`gui::Canvas`, `gui::Window`, `gui::MenuBar`, `gui::Slider`, `gui::ComboBox`, `gui::GridLayout`, `gui::Shape`, `gui::DrawableString`) - used for the entire application window, controls, and custom drawing.

Everything else (the actual cost function, Jacobians, weighting logic, DOP computation) was implemented manually.

8 Conclusion

The application successfully compares Gauss-Newton, gradient descent, and full Newton on the weighted GNSS positioning problem, both visually (through the animated convergence plot) and statistically (through the batch experiment). As expected from the theory, Gauss-Newton converges much faster than gradient descent for this type of well-conditioned least squares problem, and full Newton adds essentially nothing over Gauss-Newton, because the second-order curvature term is negligible at GPS slant-range scales. The mismatched weighting strategy (`ElevationSquared`) generally results in worse final accuracy compared to the strategy that matches the true noise model (`TrueElevation`), which illustrates why weighting choice matters in practice, not just the choice of optimization algorithm.